

ENTERPRISE KUBERNETES MASTERY

AI Platform Engineering Handbook

PART II · LINUX FOUNDATIONS FOR KUBERNETES

Namespaces, cgroups, eBPF, OverlayFS, Networking & More

Volume 2 of 16 · Foundation Series

Prerequisites: Part I — Evolution of Modern Infrastructure

Edition 2025–2026

TABLE OF CONTENTS

1. Why Linux Fundamentals Matter for Kubernetes 3
2. Processes, Threads, and the Linux Scheduler 4
3. Linux Namespaces — The Isolation Primitive 7
4. Control Groups (cgroups) — Resource Governance 12
5. OverlayFS and Union Filesystems 17
6. Linux Networking Stack 20
7. iptables, nftables, and Kubernetes Network Rules .. 24
8. eBPF — The Kubernetes Networking Revolution 28
9. System Calls and Container Security 33
10. OCI Runtime Specification and runc 36
11. Container Runtime Deep Dive 39
12. Kubernetes Leverage of Linux Primitives 43
13. Troubleshooting with Linux Tools 46
14. Hands-On Exercises 49

CHAPTER 1

Why Linux Fundamentals Matter for Kubernetes

Kubernetes is not a black box. Every concept in Kubernetes — from Pod isolation to network policy enforcement to resource limits — has a direct, verifiable implementation in the Linux kernel. An architect who understands these foundations can debug production incidents that are invisible to those who treat Kubernetes as magic. They can evaluate security claims, understand performance trade-offs, and make informed decisions about CNI plugins, runtime security tools, and node configuration.

This part maps each critical Linux concept to its Kubernetes usage, providing the systems programming foundation that separates platform engineers from platform users.

Key Insight

When a Kubernetes Pod is scheduled to a node, the kubelet calls the container runtime (containerd), which calls runc, which makes a sequence of Linux system calls to create namespaces, configure cgroups, set up OverlayFS mounts, and launch the container process. Every Kubernetes abstraction ultimately decomposes into Linux kernel calls. Understanding that decomposition is mastery.

Linux Kernel Concepts — Kubernetes Mapping

Linux Concept	Kubernetes Usage	Key Files/Commands
PID namespace	Pod process isolation	/proc/[pid]/ns/pid
Network namespace	Pod IP isolation	/proc/[pid]/ns/net, ip netns
Mount namespace	Container filesystem view	/proc/[pid]/ns/mnt
UTS namespace	Pod hostname	/proc/[pid]/ns/uts
IPC namespace	Shared memory isolation	/proc/[pid]/ns/ipc
User namespace	Rootless containers	/proc/[pid]/ns/user
cgroups v2	CPU/Memory limits, QoS	/sys/fs/cgroup/
OverlayFS	Container image layers	mount -t overlay
iptables/nftables	Service networking, NetworkPolicy	iptables-save, nft list
eBPF	Cilium networking, Falco security, Pixie observability	bpftool, bpftrace
seccomp	System call filtering (Pod Security)	/proc/[pid]/status
AppArmor/SELinux	Mandatory access control	aa-status, sestatus
TLS/socket	kubelet API, etcd communication	openssl, ss, netstat

CHAPTER 2

Processes, Threads, and the Linux Scheduler

The Linux Process Model

In Linux, everything that executes is a process (or thread, which Linux implements as a lightweight process with shared address space). Each process has a unique PID, owns file descriptors, has a virtual address space, and is associated with a set of credentials (UID, GID). The process is the fundamental unit of execution that Kubernetes manipulates through container isolation.

Key process attributes relevant to Kubernetes:

- `pid`: Process ID — unique within a PID namespace (container processes have PIDs starting at 1 within their namespace, but different PIDs on the host)
- `ppid`: Parent PID — the `init` process (PID 1 in container) must handle `SIGCHLD` to reap zombie processes; a key reason Kubernetes recommends `tini` or `dumb-init`
- `uid/gid`: User and group identity — Kubernetes `SecurityContext` sets `runAsUser`; user namespaces map container UID 0 to an unprivileged host UID
- `cwd`: Current working directory — Kubernetes sets `WORKDIR` from `Dockerfile`
- `fd table`: File descriptor table — `stdin/stdout/stderr` are FDs 0/1/2; Kubernetes captures `stdout/stderr` for `kubectl` logs
- `mm_struct`: Memory map — virtual address space; foundation for memory limits enforced by cgroups memory controller
- `task_struct`: Kernel's process descriptor — contains scheduler state, cgroup membership, namespace references, signal handlers

The Completely Fair Scheduler (CFS)

Linux's default CPU scheduler is the Completely Fair Scheduler (CFS), introduced in kernel 2.6.23. CFS models CPU time as a resource to be shared proportionally among runnable tasks based on their weight (`nice` value / `cgroup cpu.shares`). Understanding CFS is essential for understanding Kubernetes CPU requests and limits.

CFS mechanism — how Kubernetes CPU resources map to kernel scheduling:

```
Virtual runtime (vruntime): CFS tracks how much CPU time each task has consumed. The task with the lowest vruntime is scheduled next (most 'unfairly treated'). CPU request → cpu.shares: 1 CPU = 1024 shares (by convention) Pod with 500m CPU request → 512 shares CFS proportionally schedules based on share ratio CPU limit → CFS bandwidth throttling: cpu.cfs_period_us = 100,000 (100ms default) cpu.cfs_quota_us = period * limit_in_cores Example: 500m limit → quota = 100ms * 0.5 = 50ms per period Container throttled to 50ms of CPU per 100ms window CRITICAL: CPU throttling is a common hidden performance problem. Container appears healthy but responses are slow due to CFS throttling. Monitor: container_cpu_cfs_throttled_seconds_total in Prometheus
```

Warning: The CPU Throttling Trap

CPU limits cause CFS bandwidth throttling, which can introduce latency spikes even when average CPU usage is well below the limit. A container using 30% CPU on average can be throttled at 100ms intervals if it has short bursts above the limit. Many teams set CPU limits equal to requests, causing unnecessary throttling. Best practice: set CPU requests (for scheduling) but carefully evaluate whether CPU limits are needed for your workload. For latency-sensitive services, consider setting no CPU limit or a very generous limit.

Threads vs. Processes in Containers

Linux threads (created with `clone(CLONE_THREAD)`) share the same address space, file descriptors, and most resources as their parent process. The cgroups pids controller limits the total number of processes AND threads in a cgroup. This is relevant to Kubernetes because thread-heavy applications (Java with large thread pools, Go goroutines with OS threads) can hit pid limits. Kubernetes exposes this via the PodSpec: `spec.containers[].resources` and node-level pid limits (`--system-reserved`, `--kube-reserved`).

CHAPTER 3

Linux Namespaces — The Isolation Primitive

Linux namespaces are the kernel mechanism that gives each container its isolated view of system resources. When the container runtime creates a container, it calls `clone()` or `unshare()` with namespace flags, creating a new isolated context. The container process sees only the resources within its namespaces.

PID Namespace

Created with: `clone(CLONE_NEWPID)`

The PID namespace gives a process a fresh PID numbering space starting at 1. The first process in the namespace becomes PID 1 — the init process for that namespace. This is why your application container's main process should be PID 1 (or use an init helper like `tini`):

```
# On the Kubernetes node, list all processes: ps aux | grep nginx # Output shows host PID:
nginx 12847 ... # Inside the nginx container, the same process has PID 1: kubectl exec -it
nginx-pod -- ps aux # Output: nginx 1 ... # Verify namespace isolation: NODE_PID=$(kubectl get
pod nginx-pod -o jsonpath='{.status.hostIP}') # The container cannot see host PIDs unless
shareProcessNamespace: true # PID 1 must handle signals correctly: # SIGTERM → graceful
shutdown (preStop hook → SIGTERM → grace period → SIGKILL) # If PID 1 doesn't handle
SIGTERM, Kubernetes waits the full terminationGracePeriodSeconds
```

Kubernetes Relevance: `shareProcessNamespace` — When set to `true` in `PodSpec`, all containers in the Pod share a single PID namespace. This enables sidecar containers to inspect the processes of the main container — used by debugging sidecars (`kubectl debug` ephemeral containers) and security monitoring agents.

Network Namespace

Created with: `clone(CLONE_NEWNET)`

The network namespace is the most complex and important namespace for Kubernetes. Each Pod gets its own network namespace containing: a loopback interface (`lo`), a virtual ethernet (`veth`) pair connecting it to the node's network, its own routing table, `iptables/nftables` rules, and a unique IP address.

Pod network namespace setup sequence:

```
1. kubelet requests pod sandbox creation from container runtime 2. containerd creates the
'pause' container (infra container) 3. Pause container creates the network namespace ■■ All
other containers in the Pod JOIN this namespace (not create new ones) 4. CNI plugin is called
(Calico / Cilium / Flannel) 5. CNI creates veth pair: veth0 (Pod namespace) ↔ veth1 (host
namespace) 6. CNI assigns Pod IP to veth0 in Pod namespace 7. CNI configures routing: Pod IP →
veth1 → host routing → node network 8. All containers in the Pod share the same network
namespace ■■ They communicate via localhost, share the same IP and ports
```

The Pause Container: The pause container (`k8s.gcr.io/pause`) is the invisible infrastructure container that holds the Pod's network and IPC namespaces. It runs a minimal C program that simply sleeps, but it's the anchor for all namespace sharing within a Pod. If a container crashes and is restarted, it re-joins the pause

container's namespaces, preserving the Pod's IP address.

Network Namespace Inspection

```
# List network namespaces on a Kubernetes node: ip netns list # Each pod appears as:
cni-XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX # Enter a pod's network namespace and inspect:
POD_PID=$(crictl inspect $(crictl pods --name nginx -q) | \ jq -r '.status.pid') nsenter -t
$POD_PID -n ip addr show nsenter -t $POD_PID -n ip route show nsenter -t $POD_PID -n ss -tlnp #
Observe iptables rules generated by kube-proxy for Services: iptables-save | grep KUBE-SVC
```

Mount Namespace

Created with: `clone(CLONE_NEWNS)`

The mount namespace isolates the filesystem view of the container. When a container starts, the container runtime sets up an OverlayFS mount combining the container image layers with a writable layer, then pivots the container's root to this mount. The container sees only its own filesystem, not the host filesystem.

Mount namespace filesystem layout:

```
Host filesystem: Container filesystem (via overlay mount): / / etc/ etc/ ← from
image layer proc/ proc/ ← new procfs mount sys/ sys/ ← new sysfs
(restricted) var/lib/containerd/ var/ ← writable layer (image layers)
app/ ← from image layer mnt/user-data/ data/ ← from PVC mount (PV data) (bind
mount from PVC) Kubernetes volume types and their mount namespace implementation: ConfigMap →
tmpfs mount + projected files Secret → tmpfs mount (in-memory, not on disk) PVC → bind mount
from CSI driver mount point HostPath → bind mount from host path (use with extreme caution)
emptyDir → tmpfs or node-local directory bind mount
```

User Namespace (Rootless Containers)

Created with: `clone(CLONE_NEWUSER)`

User namespaces map user and group IDs between the container and the host. This enables a critical security feature: running containers as 'root' inside the container while mapping that root to an unprivileged user ID on the host. A container process that escapes its namespace cannot access host resources because it is actually a non-privileged host user.

UID/GID mapping example:

```
# Container sees UIDs 0-65535 mapped to host UIDs 100000-165535: cat
/proc/[container-pid]/uid_map # Output: 0 100000 65536 # Meaning: container UID 0 (root) =
host UID 100000 (unprivileged) # Kubernetes 1.25+ supports user namespaces in alpha: # spec: #
hostUsers: false # Enable user namespace for this Pod # Without user namespaces (current
default): # Container running as UID 0 = HOST root (dangerous if container escapes!) # This is
why securityContext.runAsNonRoot: true is critical
```


CHAPTER 4

Control Groups (cgroups) — Resource Governance

Control Groups (cgroups) are the Linux kernel mechanism for limiting, accounting for, and isolating resource usage of process groups. Kubernetes uses cgroups as the enforcement mechanism for every resource request and limit defined in a Pod spec. Without cgroups, Kubernetes resource management would be advisory only.

cgroups v1 vs. cgroups v2

cgroups v1 (legacy) used separate subsystem hierarchies for each resource type, leading to inconsistencies and management complexity. cgroups v2 (unified hierarchy, kernel 4.5+, default in major distros since 2020) provides a single, coherent hierarchy with improved resource accounting and support for delegation to unprivileged users.

Feature	cgroups v1	cgroups v2
Hierarchy	Per-subsystem (cpu, memory, blkio separate)	Single unified hierarchy
CPU accounting	cpu.shares (proportional)	cpu.weight + cpu.max (bandwidth)
Memory accounting	memory.limit_in_bytes	memory.max, memory.high
I/O control	blkio controller (limited)	io controller (complete)
Delegation	Difficult, security issues	Safe delegation to unprivileged users
Kubernetes default	Pre-2022 clusters	1.25+ with systemd, modern kernels
Rootless containers	Problematic	Fully supported

Kubernetes QoS Classes and cgroup Hierarchy

Kubernetes assigns a Quality of Service (QoS) class to every Pod based on its resource specification. This class determines the cgroup priority and eviction behaviour:

QoS Class	Condition	cgroup Priority	Eviction Priority	OOM Score
Guaranteed	requests == limits for all resources	Highest	Last evicted	-997 (protected)
Burstable	requests < limits OR partial specs	Medium	Middle	Based on usage/request ratio
BestEffort	No requests or limits set	Lowest	First evicted	1000 (first killed)

cgroups Filesystem Layout for a Kubernetes Pod

```
# cgroups v2 hierarchy on a Kubernetes node (systemd cgroup driver): /sys/fs/cgroup/
system.slice/ ← system services
kubepods.slice/ ← ALL Kubernetes pods
kubepods-besteffort.slice/ ← kubepods-pod.slice/ ← BestEffort pod
cpu.max ← no limit
memory.max ← no limit
kubepods-burstable.slice/
kubepods-pod.slice/
cpu.max ← limit in microseconds
cpu.weight ← request as weight
memory.max ← limit in bytes
kubepods-guaranteed.slice/
kubepods-pod.slice/
cpu.max ← limit == request
memory.max ← limit == request #
Inspect a specific pod's memory limit: POD_UID='xxxxxxxx-xxxx-xxxx-xxxxxxxxxxxx' cat
/sys/fs/cgroup/kubepods.slice/kubepods-burstable.slice/\
kubepods-pod${POD_UID}.slice/memory.max
```

Memory Management and OOM Killing

When a container exceeds its memory limit, the Linux Out-of-Memory (OOM) killer selects a process to kill. Kubernetes interacts with the OOM killer through OOM score adjustments:

```
# OOM score adjustment values (higher = killed first): # kubelet sets oom_score_adj for
Guaranteed pods: -997 # kubelet sets oom_score_adj for BestEffort pods: 1000 # Burstable pods:
proportional to usage/request ratio # Observe current OOM scores: cat /proc/$(pgrep -n
nginx)/oom_score_adj # memory.high (cgroups v2 only) – soft limit before hard kill: #
Container gets SIGXFSZ when usage exceeds memory.high # This is the 'memory throttle' –
application can respond before OOM kill # Kubernetes exposes this via:
resources.requests.memory (→ memory.high)
```

CHAPTER 5

OverlayFS and Union Filesystems

Container images are built from layers. When a container runs, these layers are stacked together using a union filesystem to present a single coherent filesystem to the container. OverlayFS is the default union filesystem implementation used by containerd and Docker on modern Linux systems.

OverlayFS Architecture

OverlayFS presents a merged view of two or more directories (layers):

```
OverlayFS = lower dirs (read-only) + upper dir (read-write) + work dir
Container image layers (read-only, from registry pull):
Layer 5: /bin/myapp (application binary) ← upper-most image layer
Layer 4: /etc/nginx/nginx.conf
Layer 3: /usr/sbin/nginx
Layer 2: apt-get install nginx (library files)
Layer 1: debian:bookworm base image ← lower-most image layer
Container writable layer (upper dir, per-container): /tmp/container-writes /var/log/app.log
Merged view (what container sees at /): All image layers stacked + writable layer on top
Writes go to upper dir (copy-on-write)
Deleted files: whiteout files in upper dir mask lower files #
Inspect overlay mounts on a node: mount | grep overlay # Output: overlay on / type overlay (rw,lowerdir=L5:L4:L3:L2:L1,...) # containerd stores layers at: ls /var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/snapshots/
```

Copy-on-Write (CoW) Mechanics

When a container process modifies a file that exists in a read-only lower layer, OverlayFS performs a copy-up operation: the file is copied from the lower layer to the upper (writable) layer before the modification. The container sees the modified version; the original image layer is unchanged.

Copy-on-write implications for Kubernetes:

- **Image layer sharing:** Multiple Pods using the same base image share the read-only layers on disk. A node running 100 nginx:alpine Pods stores the nginx image only once, not 100 times.
- **Write performance:** The first write to any file in a lower layer triggers a copy-up. For large files (e.g., modifying a big config file), this can cause latency. Use volumes (emptyDir, PVC) for write-heavy paths.
- **Container image size vs. runtime size:** A container's writable layer grows as it writes files. This is not tracked in the image size, only in ephemeral storage (controlled by `resources.limits.ephemeral-storage`).
- **Pod ephemeral storage limit:** Kubernetes enforces ephemeral-storage limits on the writable layer + emptyDir volumes. Exceeding this evicts the Pod.

CHAPTER 6

Linux Networking Stack

Kubernetes networking is built entirely on Linux networking primitives: virtual ethernet pairs (veth), network namespaces, bridges, routing tables, and packet filtering frameworks. Understanding these primitives explains how CNI plugins work, how Services are implemented, and why certain performance characteristics exist.

Virtual Ethernet Pairs (veth)

A veth pair is a virtual network cable: two virtual network interfaces connected to each other. Packets entering one end emerge from the other. CNI plugins use veth pairs to connect Pod network namespaces to the host network:

```
Pod network namespace Host network namespace
eth0 (Pod IP) veth0 (host end)
10.244.1.5/24 veth (no IP, in bridge) pair cni0 (bridge)
10.244.1.1/24 eth0 (node NIC) 192.168.1.10/24
# Create a veth pair (CNI does this automatically): ip link add
veth0 type veth peer name veth1 ip link set veth1 netns ip addr add 10.244.1.1/24 dev cni0 ip
link set veth0 master cni0
```

Linux Network Bridge

A network bridge is a virtual Layer 2 switch. On a Kubernetes node with Flannel or simple CNI plugins, a bridge (cni0 or cbr0) connects all Pod veth pairs. The bridge forwards packets between Pods on the same node (L2 forwarding) and routes packets to other nodes via the host routing table.

Packet Flow: Pod-to-Pod Same Node

```
Pod A (10.244.1.5) → Pod B (10.244.1.6), same node: 1. Pod A sends packet: src=10.244.1.5,
dst=10.244.1.6 2. Pod A kernel routes: dst in 10.244.1.0/24 → via eth0 (veth inside pod) 3.
Packet crosses veth pair → enters host namespace at host-side veth 4. Bridge (cni0) sees
destination MAC for 10.244.1.6 5. Bridge forwards to Pod B's veth pair host-side 6. Packet
crosses veth pair → arrives at Pod B's eth0 7. Pod B receives packet No kernel routing lookup
required for same-node traffic with bridge CNI. Cilium eBPF bypasses bridge entirely for even
lower latency.
```

Packet Flow: Pod-to-Pod Cross-Node

```
Pod A (node1: 10.244.1.5) → Pod B (node2: 10.244.2.7): FLANNEL (VXLAN overlay) path: 1. Pod A:
src=10.244.1.5, dst=10.244.2.7 2. Node1 routing: 10.244.2.0/24 → flannel.1 (VXLAN interface)
3. flanneld encapsulates: UDP VXLAN, outer dst=node2-IP:8472 4. Physical network delivers UDP
packet to node2 5. Node2 flannel.1 decapsulates → 10.244.2.7 routed to Pod B veth 6. Pod B
receives original packet CILIUM (eBPF, direct routing) path: 1. Pod A: src=10.244.1.5,
dst=10.244.2.7 2. eBPF XDP hook at veth: direct lookup in BPF map 3. BPF map shows
10.244.2.0/24 → node2-MAC via physical NIC 4. Packet forwarded directly (no VXLAN overhead) 5.
Node2 eBPF XDP: direct delivery to Pod B 6. Latency: ~20-40% lower than VXLAN
```

CHAPTER 7

iptables, nftables, and Kubernetes Network Rules

Kubernetes Services implement load balancing and service discovery using iptables (or IPVS) rules on every node. Understanding iptables is essential for troubleshooting Service connectivity, NetworkPolicy enforcement, and understanding why kube-proxy exists.

iptables Architecture

iptables processes packets through a series of tables and chains. Each packet traverses the chains in sequence; rules are evaluated top-to-bottom until a matching rule's target is applied (ACCEPT, DROP, REDIRECT, DNAT, etc.):

```
Tables and chains (traversal order for incoming packets): PREROUTING → INPUT (for local) or
FORWARD (for routing) ■■■ raw table: connection tracking exemptions ■■■ mangle table: packet
modification ■■■ nat table: DNAT (destination NAT) ← Kubernetes Services use this POSTROUTING
(for outgoing/forwarded packets) ■■■ mangle table ■■■ nat table: SNAT/MASQUERADE ←
Pod-to-external traffic Kubernetes KUBE-* chains (inserted by kube-proxy): PREROUTING nat →
KUBE-SERVICES KUBE-SERVICES → KUBE-SVC-XXXXXXXXXXXXXXXXX (per Service) KUBE-SVC-XXX →
KUBE-SEP-YYYYYYYYYYYYYYYYY (per Endpoint, with probability) KUBE-SEP-YYY → DNAT to Pod IP:Port
# View Service iptables rules: iptables-save -t nat | grep KUBE-SVC iptables -t nat -L
KUBE-SERVICES -n --line-numbers | head -30
```

How Kubernetes Service IP Works

A Kubernetes Service ClusterIP (e.g., 10.96.50.100) is a virtual IP — it does not exist on any network interface anywhere in the cluster. It is purely an iptables DNAT rule that rewrites destination IP from the ClusterIP to a Pod IP before the packet leaves the node's networking stack.

```
Service ClusterIP → Pod IP translation: Pod sends: src=10.244.1.5, dst=10.96.50.100:80
(ClusterIP) iptables DNAT (random load balancing across 3 endpoints): 33% probability: DNAT to
10.244.1.7:8080 (pod-1) 33% probability: DNAT to 10.244.2.3:8080 (pod-2) 33% probability: DNAT
to 10.244.3.9:8080 (pod-3) Packet after DNAT: src=10.244.1.5, dst=10.244.2.3:8080 Forwarded to
node2 via CNI overlay/routing Return packet: src=10.244.2.3:8080, dst=10.244.1.5 conntrack:
automatically SNAT back to 10.96.50.100:80 Pod sees: src=10.96.50.100:80, dst=10.244.1.5
(consistent with request)
```

iptables at Scale — The kube-proxy Problem

iptables has a critical scalability limitation: rules are evaluated linearly. With 10,000 Services and 100,000 endpoints, a packet must traverse up to 200,000 iptables rules before finding a match. This causes $O(n)$ latency that grows with cluster size. Two solutions:

- **IPVS mode:** kube-proxy in IPVS mode uses kernel virtual server (IPVS) hash tables for $O(1)$ service lookup, scaling to hundreds of thousands of services with minimal latency.
- **Cilium eBPF:** Replaces kube-proxy entirely with eBPF maps that provide $O(1)$ lookup without iptables traversal. The definitive solution for large clusters.

CHAPTER 8

eBPF — The Kubernetes Networking Revolution

Extended Berkeley Packet Filter (eBPF) is a revolutionary Linux kernel technology that allows running sandboxed programs in the kernel without modifying kernel source code or loading kernel modules. eBPF is transforming Kubernetes networking, security, and observability in ways that were impossible with traditional kernel mechanisms.

What eBPF Is and How It Works

eBPF programs are small, verified programs attached to kernel hook points. When a hook point fires (e.g., a packet arrives at a network interface, a system call is made, a kernel function is called), the eBPF program runs in a restricted virtual machine within the kernel context — with direct access to kernel data structures but without the risk of crashing the kernel.

```
eBPF program lifecycle: 1. Write eBPF program in C (or Rust) – restricted subset 2. Compile to eBPF bytecode: clang -target bpf -O2 prog.c -o prog.o 3. Load into kernel via bpf() syscall 4. Kernel verifier checks: no loops, bounded memory access, safe termination 5. JIT-compile to native CPU instructions (x86, ARM64) 6. Attach to hook point: XDP, tc, kprobe, tracepoint, socket 7. Program runs at hook point with ~nanosecond overhead 8. eBPF maps enable data sharing between kernel eBPF and userspace # eBPF hook points relevant to Kubernetes: XDP (eXpress Data Path): packet arrival at NIC, before kernel networking TC (Traffic Control): packet after NIC driver, before routing kprobe/kretprobe: kernel function entry/exit (observability) uprobe: userspace function entry/exit tracepoint: kernel static instrumentation points LSM (Linux Security): security policy enforcement hooks
```

eBPF in Kubernetes — Key Use Cases

Domain	Tool	eBPF Usage	Benefit over Legacy
Networking	Cilium	Replace kube-proxy with eBPF maps for O(1) service routing	10-40% latency reduction, massive scale
Networking	Cilium	NetworkPolicy enforcement in kernel without iptables	Lower latency, richer policy (L7)
Networking	Cilium	Transparent service mesh with eBPF sockops	No sidecar proxies, 30% less overhead
Security	Falco	Kernel syscall monitoring via eBPF (replaces kernel module)	Safer, portable, lower overhead
Security	Tetragon	Runtime security policy enforcement at kernel level	Block attacks before they land
Observability	Pixie	Auto-instrumentation via uprobes — no code changes needed	Zero-code tracing for Go, Java, Python

Domain	Tool	eBPF Usage	Benefit over Legacy
Observability	Cilium Hubble	Network flow visibility via eBPF	Full L3-L7 visibility, no sampling
Performance	BCC/bpftrace	Production profiling: CPU flamegraphs, latency heatmaps	Nanosecond precision, no overhead

Cilium — eBPF-Native Kubernetes Networking

Cilium is the most widely adopted eBPF-based CNI for Kubernetes. It replaces the traditional networking stack (iptables + veth + bridge) with eBPF programs that implement service routing, load balancing, and network policy entirely within the kernel data plane.

```
Cilium architecture replacing traditional kube-proxy + CNI: Traditional: Pod → veth → bridge → iptables NAT → physical NIC (5-8 kernel operations per packet) Cilium eBPF: Pod → eBPF TC hook → BPF map lookup → physical NIC (1-2 operations per packet, with XDP: even at NIC driver level) Service routing without kube-proxy: ClusterIP → BPF SNAT/DNAT maps (O(1) hash lookup) NodePort → BPF handled at NIC level (XDP) LoadBalancer → BPF DSR (Direct Server Return) – response bypasses LB # Enable Cilium with kube-proxy replacement: helm install cilium cilium/cilium \ --set kubeProxyReplacement=strict \ --set k8sServiceHost= \ --set k8sServicePort=6443
```

CHAPTER 9

System Calls and Container Security

System calls are the interface between user-space processes and the Linux kernel. Every container operation — opening files, network sockets, creating processes, allocating memory — ultimately invokes a system call. Restricting the set of system calls available to a container is a critical security defence layer.

seccomp — System Call Filtering

seccomp (Secure Computing mode) is a Linux kernel feature that restricts the system calls a process can make. Kubernetes supports seccomp profiles through the `securityContext.seccompProfile` field:

```
# Apply the default container runtime seccomp profile: securityContext: seccompProfile: type:
RuntimeDefault # Blocks ~50 dangerous syscalls # Apply a custom seccomp profile (stored as
ConfigMap or node file): securityContext: seccompProfile: type: Localhost localhostProfile:
profiles/nginx-seccomp.json # Example seccomp profile blocking dangerous syscalls: {
"defaultAction": "SCMP_ACT_ERRNO", "architectures": ["SCMP_ARCH_X86_64"], "syscalls": [{
"names": ["read", "write", "open", "close", "stat", "fstat", "lstat", "poll", "mmap",
"mprotect", "munmap", ...], "action": "SCMP_ACT_ALLOW" }] }
```

Linux Capabilities — Privilege Decomposition

Historically, Linux had two privilege levels: root (all privileges) and non-root (minimal privileges). Linux capabilities decompose root privileges into 40+ individual capabilities that can be granted or revoked independently. Kubernetes SecurityContext allows fine-grained capability management:

Capability	What It Allows	Kubernetes Implication
CAP_NET_ADMIN	Configure network interfaces, routes, iptables	Privileged networking; required for CNI plugins on nodes
CAP_SYS_ADMIN	Wide range of system admin operations	Nearly root — almost always indicates container escape risk
CAP_NET_RAW	Raw socket operations, packet sniffing	Required for ping; also enables ARP spoofing attacks
CAP_SYS_PTRACE	Trace and inspect other processes	Required for some debuggers; security risk
CAP_CHOWN	Change file ownership	Usually unnecessary for application containers
CAP_SETUID	Change user ID to any UID	Privilege escalation risk — drop unless required
CAP_NET_BIND_SERVICE	Bind to ports below 1024	Better: run on high port, use Service for port mapping

```
# Security-hardened container spec: securityContext: runAsNonRoot: true runAsUser: 1000
runAsGroup: 1000 readOnlyRootFilesystem: true allowPrivilegeEscalation: false capabilities:
drop: - ALL # Drop all capabilities add: - NET_BIND_SERVICE # Add only what's needed
```



```
seccompProfile: type: RuntimeDefault
```

CHAPTER 10

OCI Runtime Specification and runc

The OCI Runtime Specification defines the interface between a container image (as an extracted filesystem bundle) and the container runtime. `runc` is the reference implementation — it is the component that ultimately makes the Linux system calls to create the container process.

The Container Creation Sequence

```
Kubernetes → kubelet → containerd → runc → Linux kernel: 1. API Server: Pod manifest
accepted, written to etcd 2. Scheduler: Pod bound to node 3. kubelet: watches for Pods bound
to its node 4. kubelet: calls containerd CRI RunPodSandbox 5. containerd: creates pause
container (network/IPC namespace holder) 6. containerd: calls CNI plugin to configure Pod
network namespace 7. kubelet: calls containerd CRI CreateContainer for each container 8.
containerd: pulls image if not cached; prepares overlay snapshot 9. containerd calls runc with
OCI bundle (config.json + rootfs) 10. runc reads config.json (namespaces, cgroups, mounts,
capabilities) 11. runc executes: a. clone() with namespace flags b. Join cgroup hierarchy c.
Set up mounts (overlay, proc, sys, devpts, volumes) d. Drop capabilities, set seccomp profile
e. execve() the container entrypoint 12. Container process runs as PID 1 in its namespace 13.
kubelet: monitors via CRI ContainerStatus calls
```

runc config.json — The OCI Runtime Spec

```
# Simplified OCI config.json generated by containerd for a Pod container: { "ociVersion":
"1.0.2", "process": { "user": { "uid": 1000, "gid": 1000}, "args": ["/app/server"], "env":
["PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin"], "capabilities": { "effective": [],
// all capabilities dropped "permitted": [] }, "rlimits": [{"type": "RLIMIT_NOFILE", "hard":
65536, "soft": 65536}] }, "root": {"path": "rootfs", "readonly": true}, "mounts": [
{"destination": "/proc", "type": "proc"}, {"destination": "/sys", "type": "sysfs", "options":
["ro"]}, {"destination": "/data", "type": "bind", "source":
"/var/lib/kubelet/pods//volumes/pvc-xxx"} ], "linux": { "namespaces": [ {"type": "pid"},
{"type": "ipc"}, {"type": "uts"}, {"type": "mount"}, {"type": "network", "path":
"/proc//ns/net"} ], "cgroupsPath": "/kubepods/burstable/pod/", "seccomp": {"defaultAction":
"SCMP_ACT_ERRNO", "syscalls": [...]} } }
```

CHAPTER 11

Container Runtime Deep Dive

Container Runtime Interface (CRI)

The Container Runtime Interface (CRI) is the gRPC API that Kubernetes uses to communicate with container runtimes. Any runtime implementing CRI can serve as a Kubernetes container runtime. This decoupling enables runtime choice without Kubernetes code changes.

Runtime	Architecture	OCI Runtime	Best For	GPU Support
containerd	Daemon + snapshotter + OCI executor	runc (default), kata-runtime	General purpose, production default	NVIDIA GPU Operator, CDI
CRI-O	Lightweight, K8s-specific daemon	runc, kata	Red Hat OpenShift, minimal footprint	Same as containerd via CDI
gVisor (runsc)	Intercepting kernel sandbox	runsc (user-space kernel)	Untrusted workloads, SaaS	Limited (no CUDA)
Kata Containers	VM-based container isolation	kata-runtime → QEMU/Firecracker	High-security, multi-tenant	Kata + GPU passthrough
Firecracker	MicroVM, sub-second boot	firecracker-containerd	Serverless (AWS Lambda)	Emerging support
Podman	Daemonless, rootless	runc, crun	Developer workstations, rootless	Via CDI

Kata Containers — VM-Level Isolation in Kubernetes

Kata Containers runs each container (or Pod) inside a lightweight virtual machine, providing hypervisor-level isolation while maintaining OCI/CRI compatibility. This is the standard approach for multi-tenant Kubernetes where workloads from different tenants must not share a kernel.

```
# Enable Kata Containers via RuntimeClass: apiVersion: node.k8s.io/v1 kind: RuntimeClass
metadata: name: kata-containers handler: kata --- # Use Kata in a Pod spec: spec:
runtimeClassName: kata-containers containers: - name: untrusted-workload image:
customer-code:latest # Trade-offs vs standard runc: Isolation: Hypervisor boundary (much
stronger) Startup: +200-500ms (VM boot vs process fork) Memory: +50-100MB per Pod (VM
overhead) Performance: ~5-10% CPU overhead vs runc
```

CHAPTER 12

Kubernetes Leverage of Linux Primitives

Kubernetes is a high-level orchestration API built entirely on Linux primitives. This chapter provides the definitive mapping from Kubernetes abstractions to their Linux kernel implementation.

Kubernetes Concept	Linux Primitive	Implementation Detail
Pod isolation	PID + Net + Mnt + UTS + IPC namespaces	6 namespaces per Pod sandbox (pause container holds net/ipc)
Pod IP address	Network namespace + veth pair + CNI	Each Pod gets dedicated netns; CNI assigns IP via veth
CPU requests	CFS cpu.weight (cgroups v2)	Relative scheduling weight; prevents starvation
CPU limits	CFS bandwidth (cpu.max)	Throttles to N ms per period; source of latency spikes
Memory limits	cgroups memory.max	Hard limit; exceeding triggers OOM kill of container process
Memory requests	cgroups memory.high + scheduler	Scheduler uses for node placement; memory.high soft limit
Service ClusterIP	iptables DNAT / eBPF BPF map	Virtual IP implemented as packet rewrite rule on every node
NetworkPolicy	iptables / eBPF / Calico Felix	Per-pod L3/L4 ACLs; eBPF enables L7 policies
Volume (emptyDir)	tmpfs mount / bind mount	tmpfs for memory-backed; node directory for disk-backed
Volume (PVC)	CSI mount + bind mount into netns	CSI driver mounts device/NFS; kubelet bind-mounts into Pod netns
ConfigMap/Secret	tmpfs + projected volume	Files are tmpfs-backed; Secrets never touch disk (in-memory)
Container image	OverlayFS layers	Lower dirs = image layers; upper dir = writable container layer
runAsNonRoot	setuid() / setgid()	runc sets UID/GID before execve(); kernel enforces
capabilities	Linux capabilities (cap_set_proc)	runc drops all caps then adds only specified ones
seccompProfile	seccomp BPF filter	BPF filter attached to process; blocks specified syscalls
AppArmor	Linux Security Module (LSM)	Per-container AppArmor profile loaded from node

Kubernetes Concept	Linux Primitive	Implementation Detail
QoS Guaranteed	cgroup oom_score_adj = -997	Protected from OOM kill; last evicted under node pressure
Node affinity	Scheduler + kubelet node labels	Scheduler uses topology info; no kernel primitive involved
GPU resources	cgroups devices + NVIDIA drivers	Device plugin exposes GPU via CDI; cgroup controls access

CHAPTER 13

Troubleshooting with Linux Tools

Essential Diagnostic Command Reference

Namespace and Container Inspection

```
# Get Pod PID on node: crictl pods --name crictl inspect | jq '.status.pid' # Enter Pod
network namespace (diagnose connectivity): nsenter -t -n -- ip addr nsenter -t -n -- ip route
nsenter -t -n -- ss -tlnp nsenter -t -n -- iptables-save # Enter Pod mount namespace (inspect
filesystem): nsenter -t -m -- ls -la / nsenter -t -m -- cat /proc/mounts # Inspect cgroup
limits for a Pod: CGPATH=/sys/fs/cgroup/kubepods.slice/kubepods-burstable.slice/ cat
$CGPATH/kubepods-pod.slice/memory.max cat $CGPATH/kubepods-pod.slice/cpu.max # Check CPU
throttling: cat $CGPATH/kubepods-pod.slice/cpu.stat | grep throttled
```

eBPF-Based Observability

```
# Trace all exec() calls on the node (detect unexpected processes): bpftrace -e
'tracepoint:syscalls:sys_enter_execve { printf("%s -> %s\n", comm, str(args->filename)); }' #
Trace TCP connections established by Pods: bpftrace -e 'tracepoint:tcp:tcp_connect {
printf("pid=%d comm=%s\n", pid, comm); }' # CPU flamegraph using BCC perf tools: profile -F 99
-p 30 > out.txt flamegraph.pl out.txt > flamegraph.svg # Pixie: zero-instrumentation
observability for K8s # Automatically instruments Go, Java, Python, Node via uprobes px run
px/service_stats -- --service=
```

Common Linux-Level Kubernetes Issues

Issue: Pod OOMKilled

Symptom: Container exits with OOM kill

Diagnosis & Fix: Check memory.max vs usage: `kubectl top pod` + node event logs. Increase memory limit or reduce application memory usage.

Issue: CPU Throttling

Symptom: Service latency without high CPU usage

Diagnosis & Fix: Check `container_cpu_cfs_throttled_seconds_total` metric. Increase CPU limit or remove it; verify CPU requests match actual usage.

Issue: DNS resolution failures

Symptom: Pods cannot resolve service names

Diagnosis & Fix: `nsenter` into pod netns: `nslookup kubernetes.default`. Check CoreDNS pods, `ndots` setting, search domains.

Issue: iptables rule exhaustion

Symptom: Service connectivity intermittent

Diagnosis & Fix: Check iptables rule count: `iptables -L | wc -l`. Consider IPVS mode or Cilium if >10,000 services.

Issue: OverlayFS disk full

Symptom: Container cannot write files

Diagnosis & Fix: Check ephemeral storage: `kubectl describe pod`. `df -h /var/lib/containerd`. Consider `emptyDir` with `sizeLimit`.

Issue: Zombie processes

Symptom: PID namespace fills with defunct processes

Diagnosis & Fix: Application not reaping children (PID 1 problem). Add `tini` or `dumb-init` as PID 1 in container.

CHAPTER 14

Hands-On Exercises

Exercise 2.1 — Namespace Exploration

Deploy a test Pod and directly inspect its Linux namespace isolation:

```
kubectl run debug-pod --image=nginx:alpine --restart=Never kubectl wait --for=condition=Ready
pod/debug-pod # Find Pod PID on node (SSH to node or use node debug): kubectl debug
node/NODE_NAME -it --image=ubuntu -- bash # Use crictl to find pod PID: POD_ID=$(crictl pods
--name debug-pod -q | head -1) POD_PID=$(crictl inspect $POD_ID | python3 -c 'import sys,json;
d=json.load(sys.stdin); print(d["status"]["pid"])') # List all namespaces of the Pod: ls -la
/proc/$POD_PID/ns/ # Compare host vs container PID namespaces: ls -la /proc/1/ns/pid # host
init ls -la /proc/$POD_PID/ns/pid # pod -- different inode # Inspect network namespace:
nsenter -t $POD_PID -n ip addr nsenter -t $POD_PID -n ip route nsenter -t $POD_PID -n ss -tlnp
```

Exercise 2.2 — cgroups Resource Enforcement

Observe how Kubernetes CPU and memory limits translate to cgroup values:

```
# Deploy Pod with resource limits (save as resource-demo.yaml and kubectl apply): apiVersion:
v1 kind: Pod metadata: name: resource-demo spec: containers: - name: app image: nginx:alpine
resources: requests: cpu: 250m memory: 128Mi limits: cpu: 500m memory: 256Mi # Find the cgroup
path on the node: POD_UID=$(kubectl get pod resource-demo -o jsonpath=.metadata.uid)
CGPATH=/sys/fs/cgroup/kubepods.slice/kubepods-burstable.slice cat
$CGPATH/kubepods-pod-POD_UID.slice/memory.max # Expected: 268435456 (256 x 1024 x 1024) cat
$CGPATH/kubepods-pod-POD_UID.slice/cpu.max # Expected: 50000 100000 (50ms quota per 100ms
period = 500m CPU)
```

Exercise 2.3 — eBPF Observability

Use eBPF tools to observe container behaviour:

```
# Install bpftrace on a Kubernetes node: apt-get install -y bpftrace # Observe all exec()
syscalls (detect unexpected binaries running in pods): bpftrace -e '
tracepoint:syscalls:sys_enter_execve { printf("%d %-20s %s\n", pid, comm,
str(args->filename)); }' # Observe all TCP connections established from pods: bpftrace -e '
tracepoint:sock:inet_sock_set_state /args->newstate == 1/ { printf("CONNECT pid=%d comm=%s\n",
pid, comm); }' # Generate CPU flamegraph for a running pod: # (requires Linux perf +
flamegraph.pl) perf record -F 99 -p $POD_PID -g -- sleep 30 perf script |
stackcollapse-perf.pl | flamegraph.pl > cpu.svg
```

End of Part II — [Continue to Part III: Containers](#)

Part III provides a comprehensive deep-dive into container technologies: Docker architecture, OCI specifications, containerd and CRI-O, image building best practices (distroless, multi-stage, scratch images), secure supply chain (Cosign, SBOM, SLSA), and runtime security for production environments.